

SparSEEtY: Extracting Tokens from Sparsity-Exploiting LLM Serving Systems via Deterministic Side Channels

Yongwan Jo*, Jinyoung Park*, Euihyun Lee, Dokyung Song†
Department of Computer Science
Yonsei University

Abstract—Modern large language models (LLMs) exhibit *activation sparsity*, wherein only a subset of their neurons is activated for given input tokens. Researchers have leveraged this property to optimize LLM serving systems by omitting weight accesses and computations pertaining to inactive neurons. Unfortunately, however, such optimizations create input-dependent weight accesses, which can be leaked over side channels.

We present SparSEEtY, a new token extraction attack that exploits input-dependent neuron weight accesses introduced by sparsity-exploiting LLM serving systems. SparSEEtY first constructs a neuron-activation oracle using neuron weight access side channels during LLM inference, and then inverts the activation traces to reconstruct the input tokens, forming an end-to-end token extraction attack. We instantiate SparSEEtY against an LLM serving system protected inside an Intel TDX confidential virtual machine (CVM), addressing three key challenges: (i) constructing a neuron-activation oracle using a combination of side channels exposed by CVMs, (ii) reducing inference-time overheads of neuron activation monitoring for covertness, and (iii) accurately inverting partial binary activation traces back to tokens. Our evaluation shows that SparSEEtY can reconstruct both prompt and response tokens with consistently high BLEU scores (>0.95) across various models and datasets, while incurring monitoring overheads of 3.7% to 7.2%.

I. INTRODUCTION

Modern large language models (LLMs) exhibit *activation sparsity*—a phenomenon in which only a subset of neurons is activated during inference depending on the input tokens. This property offers both interpretability benefits, e.g., yielding more structured internal representations [1] and improved model alignment [2], as well as significant efficiency gains: Since only a small fraction of neurons in a Transformer [3] model’s Feed-Forward Networks (FFNs) is activated in response to a given sequence of tokens, previous work has proposed to skip memory accesses and disk block accesses associated with inactive neurons [4]–[6]. The resulting sparse computation significantly reduces both compute and memory overheads, enabling larger LLMs to be served with fewer resources [6]–[8].

However, this efficiency is gained at the cost of new privacy risks: Namely, the memory access patterns resulting from

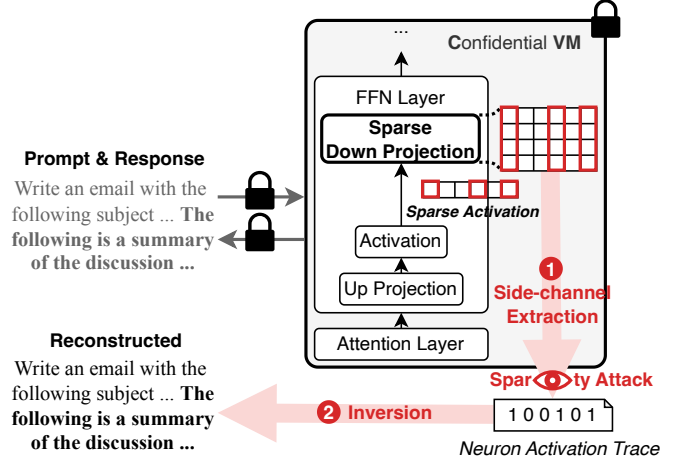


Fig. 1: High-level overview of the SparSEEtY attack.

sparse neuron computation become dependent on the input. In many adversarial settings, these patterns can be leaked over side channels, e.g., via CPU cache monitoring [9], page fault monitoring [10], or even physical address bus probing [11]–[13]. Then, an attacker who observes these patterns could potentially correlate them with the input token sequence and infer sensitive information contained within it.

This paper concretely demonstrates the privacy risks of sparse neuron computation by presenting SparSEEtY, an end-to-end token extraction attack against sparsity-exploiting LLM serving systems. As illustrated in Fig. 1, SparSEEtY targets a Transformer-based LLM deployed inside a Confidential Virtual Machine (CVM), where users expect strong confidentiality guarantees for their prompts and generated responses. In this setting, a user establishes a secure end-to-end communication channel with a CVM-hosted LLM service, and submits a prompt. The prompt tokens, along with autoregressively generated tokens, pass through the Transformer layers, and each FFN performs sparse down projection based on sparse activation. The generated tokens are returned back to the user as a response through the same secure channel.

The SparSEEtY attack, operating under the standard CVM adversary model, proceeds in two phases. First, during the

*Equal contribution.

†Corresponding author.

LLM’s inference on a victim’s prompt, SparSEETy captures neuron activation patterns by monitoring selective accesses to down projection weights via a page fault side channel (also known as the controlled channel [10]). SparSEETy then inverts the captured neuron activation traces to reconstruct the tokens constituting the user’s original prompt, as well as the LLM-generated response.

SparSEETy addresses three key challenges. First, building a neuron-activation oracle from the memory-access oracle provided by page fault side channels requires identifying the guest physical addresses (GPAs) of each neuron’s down projection weights, which reside in guest-private memory. We achieve this by combining page fault side channels with two additional deterministic side channels: block I/O and page allocation side channels. SparSEETy detects when the victim reads the target weights from its block device, and then monitors subsequent accesses to recently allocated pages to efficiently establish the weight-to-GPA mappings.

Second, naive use of page fault side channels could incur costly CVM exits during victim inference, creating both performance overhead and detection risk. We address this with two techniques: selective neuron monitoring and single-step-free chained page arming. For selective monitoring, we apply an information-theoretic method [14] to identify a small subset of neurons that retain maximal discriminative power. Our chained page arming technique further eliminates unnecessary exits caused by redundant page faults and single-stepping, while precisely capturing activation traces for all monitored neurons across all token positions.

Third, unlike prior inversion work [15]–[17] that assumes access to full-precision activation values, our oracle reveals only binary activations (i.e., activated or not), and only for a subset of neurons due to selective monitoring. This creates ambiguity, as different tokens may produce identical binary patterns. Moreover, a naive exhaustive search over the vocabulary at each token position is computationally expensive. To address this, we guide the search using the probability distribution from a full forward pass of the LLM, conditioned on previously reconstructed tokens. Candidate words are then tested in descending order of likelihood, and the first word whose binary activation pattern matches the observation is selected. This approach significantly reduces the search overhead without compromising, and often even improving, reconstruction accuracy.

Our attack is related to (yet distinct from) prior work showing that intermediate activations can be used to reconstruct inputs. Pioneered by Mahendran and Vedaldi [15], who inverted intermediate representations of convolutional neural networks to reconstruct input images, recent studies have proposed inverting the activations of intermediate Transformer layers or even the final next token probabilities to reconstruct input tokens. These methods, however, assume either access to full-precision activation values [16], [17]—requiring the attacker to participate in inference or act as an auditor—or live query access to the victim LLM service for system prompt extraction [18]. In contrast, SparSEETy operates strictly under the standard

CVM threat model: it extracts activation values at only binary precision through our neuron activation oracle, and requires no live query access to the target LLM service.

We implemented SparSEETy atop Linux KVM [19] and QEMU [20], targeting an LLM service running PowerInfer [7] protected within an Intel TDX CVM. Our evaluation shows that SparSEETy can covertly extract neuron activation patterns with only 3.7% to 7.2% inference-time overhead across models, while accurately inverting the neuron activation traces. Notably, monitoring only 100 first-layer FFN neurons, representing just 0.015%–0.028% of all FFN neurons across layers, suffices to accurately recover both prompt and response tokens (BLEU score >0.95) across all models and datasets. SparSEETy remains effective even when a private LoRA [21] adapter unknown to the adversary is applied, with a modest increase to 400 monitored neurons (still a small fraction of all FFN neurons). More broadly, our findings expose a fundamental tension between efficiency and privacy in LLM serving: as new resource-efficient inference optimizations emerge, their privacy risks demand rigorous investigation.

In summary, we contribute the following:

- We propose a new side-channel token extraction attack against CVM-protected LLM serving systems employing sparse computation for resource-efficient inference.
- We propose a suite of techniques to (i) extract neuron activation traces from a CVM using a combination of deterministic side channels, (ii) reduce inference-time overhead caused by neuron activation monitoring for stealthy operation, and (iii) invert partial, binary activation traces to accurately reconstruct both prompt and response tokens.
- We implemented a complete prototype of our attack against an Intel TDX-protected LLM serving system, and empirically demonstrate our attack can effectively recover the tokens while ensuring covert operation.

II. BACKGROUND

A. Using Activation Sparsity in LLM Serving

Transformer-based LLMs take as input a sequence of tokens (i.e., an input prompt) denoted by $(x_1, \dots, x_n)$, where each token x_i is drawn from a fixed vocabulary $\mathcal{V}$, and generate output tokens $(x_{n+1}, \dots, x_{n+T})$ during inference [3]. These output tokens are generated autoregressively, one token at a time, until the model emits a special end-of-sequence (EOS) token (typically x_{n+T}). LLM inference proceeds in two stages: prefill and decode. In the prefill stage, the model processes the entire input prompt to compute a probability distribution $P(v|x_1, \dots, x_n) : v \in \mathcal{V}$, from which it samples the first new token x_{n+1} . In the decode stage, the model computes the probability distribution for each subsequent token autoregressively, conditioning on the input prompt and all previously generated tokens: $P(v|x_1, \dots, x_n, \dots, x_{n+i-1}) : v \in \mathcal{V}$. The model then samples x_{n+i} from this distribution, and the process continues until an EOS token is generated.

These LLMs stack multiple Transformer layers, each transforming input representations to output representations of the

same size, referred to as the hidden dimension d_{model} . Each Transformer layer includes a Feed-Forward Network (FFN) sublayer, a two-layer multi-layer perceptron comprising two linear transformations with an activation function, e.g., Rectified Linear Unit (ReLU), applied between them. The FFN first applies *up projection*, a linear layer that maps the input to a higher-dimensional space (from $\mathbb{R}^{d_{\text{model}}}$ into $\mathbb{R}^{d_{\text{ff}}}$). The result is passed through an activation function, and then projected back to d_{model} by another linear layer called *down projection*.

Dynamic Activation Sparsity refers to a phenomenon where neurons in the Transformer’s FFN sublayers are selectively activated depending on the current input [22]. The degree of sparsity varies across models, with each neuron activated for only 1.48% to 25.65% of tokens on average in ReLU and ReLU² models, according to our profiling (see §VIII).

- **Naturally-Occurring Activation Sparsity.** For models using ReLU (or its variants such as ReLU² [23]) as their activation functions, a large degree of activation sparsity naturally arises as any negative values are mapped to zero after applying the activation function.
- **Artificially-Induced Activation Sparsity.** For models employing activation functions other than ReLU variants, the proportion of zero-valued activations is typically negligible. Prior work has proposed to artificially induce dynamic activation sparsity, either by *ReLU-fying* the model ahead of time (i.e., replacing the activation function with ReLU followed by fine-tuning) [24] or by applying run-time thresholding to suppress low-magnitude activations [25].

Sparse Down Projection in FFN Sublayers. Prior work leveraged activation sparsity to perform the down projection computations in Transformer’s FFN sublayers [4], [26] via sparse matrix-vector multiplication. The core idea is that dynamic activation sparsity produces highly sparse activations within each FFN, and the columns of the down projection weight matrix corresponding to zero-valued activations (arising from ReLU-like functions and other activation functions followed by magnitude-based thresholding) can be skipped without affecting the output. A major advantage of using sparse matrix-vector multiplication is the reduction in computational cost. For instance, Mirzadeh et al. report a 32% decrease in computation for OPT-6.7b [24]. Researchers further proposed to predict which neurons will be activated before applying activation, enabling the sparse matrix-vector multiplications to be applied to the up projection as well [4].

B. Deterministic Side Channels in CVMs

We now describe deterministic side channels that we exploit in our token extraction attack. We focus on the side channels that arise in virtualization-based trusted execution environments (TEEs), also known as confidential virtual machines (CVMs).

Page Access Side Channel. TEEs commonly delegate memory management to the untrusted host for reducing the complexity and size of the trusted computing base. Exploiting this design choice, Xu et al. [10] demonstrated page fault side channel

attacks (also known as controlled-channel attacks) against Intel SGX [27], in which the host OS removes virtual-to-physical page mappings to trigger enclave page faults. Similar attacks have also been shown against CVMs [28]–[31], including those built on AMD SEV [32] and Intel TDX [33], through manipulation of mappings from guest physical addresses (GPAs) to host physical addresses (HPAs).

In Intel TDX, the trusted TDX module [34], running in a newly added CPU mode called Secure Arbitration Mode (SEAM), manages Secure Extended Page Tables (SEPTs) that contain GPA-to-HPA mappings for CVM-private pages. This module exposes a SEAMCALL interface to the untrusted host. It includes the following functions that can be used by host-side adversaries to manipulate SEPT entries and trigger page faults in CVMs, thereby creating page fault side channels.

- TDH.MEM.RANGE.BLOCK blocks guest access to specified GPA ranges, triggering page faults on subsequent accesses.
- TDH.MEM.TRACK increments the CVM’s TLB tracking counter, which is used for TLB shutdown described below.
- TDH.MEM.RANGE.UNBLOCK unblocks specified GPA ranges previously blocked, restoring guest access to them.

When the host blocks guest pages, it must perform a TDX-specific TLB shutdown to invalidate stale GPA-to-HPA mappings cached on other cores executing the same CVM. This is done by issuing inter-processor interrupts to force those cores to exit the CVM, and then invoking TDH.MEM.TRACK to increment the global counter. Each core maintains a local counter, and, upon detecting a mismatch with the global counter on its next entry into the CVM, flushes all CVM-associated TLB entries. Once a GPA range is blocked, accesses trigger page faults without populating the TLB; consequently, unblocking the range does not require a shutdown, and normal TLB caching for the unblocked range resumes.

Page Allocation Side Channel. Intel TDX supports demand paging for efficient memory management. When a CVM accesses unallocated pages, it triggers a CVM exit with an SEPT fault. The host handles this by invoking TDH.MEM.PAGE.AUG, a SEAMCALL that requests additional private pages. This creates a side channel: a malicious host can observe the timing and sequence of the CVM’s page allocation.

Block I/O Side Channel. CVMs rely on block devices provided by the untrusted host virtual machine monitor (VMM) such as QEMU [35], for non-volatile storage that hosts their filesystems. CVMs access these devices by requesting sectors to the VMM, specifying (i) the sector offset and (ii) the GPA for the destination. This creates a block I/O side channel: an attacker on the host VMM side can observe (i) which sectors in the block device are requested by the CVM, and (ii) which GPAs receive them. Notably, the GPA provided by the CVM points to a bounce buffer located in its non-private memory shared with the VMM, since the VMM is not permitted to access the CVM’s private memory. Once the VMM copies the requested sectors into the bounce buffer, the CVM copies (or “bounces”) the data back to its private memory.

them back into its private memory when resumed, typically into their OS’s page cache.

III. THREAT MODEL & ASSUMPTIONS

We consider deploying an LLM service within a CVM, such as a Trust Domain in Intel TDX, reflecting growing interest in deploying DNN serving within TEEs across both academia [36]–[40] and industry [41]–[43]. We focus on private, CPU-based inference scenarios where each user provisions a dedicated CVM hosting an LLM service to securely process sensitive prompts, leveraging CPU TEEs as a secure and cost-effective alternative to GPU TEEs for moderate batch and input sizes [44].

To improve computational efficiency, the victim LLM service is assumed to employ the following optimizations: (i) model parallelism [45], where multiple CPUs are allocated to the CVM, and multiple threads are spawned within it to enable parallel execution of LLM inference, (ii) KV cache, which removes redundant computation caused by Transformer’s attention mechanism during autoregressive token generation, by caching the key (K) and value (V) vectors of all Transformer layers for all prompt tokens and previously generated tokens, and (iii) sparsity-exploiting optimizations, specifically sparse matrix-vector multiplication in the FFN layers during the down projection step, leveraging dynamic activation sparsity (see §II-A). An optimized LLM serving framework called PowerInfer [7], for example, employs all of these optimizations. In this setting, we assume a standard, host-side CVM attacker who aims to infer the tokens being processed by the victim LLM service, by exploiting side channels exposed by the CVM.

Defensive Assumptions. We assume that the CVM exposes the LLM service to legitimate users only, via an authenticated, encrypted communication channel; in other words, the attacker has no query access to the victim LLM service. This implies that prompt extraction attacks across co-tenants of the LLM service, such as PromptPeek [46], are not applicable in our setting. We also assume that known side-channel prompt extraction attacks, such as the one proposed by Gao et al. [47], which targets the victim LLM service’s embedding table access, are mitigated. This can be achieved by removing the embedding lookup accesses, e.g., through deep hash embedding techniques [48], [49].

We also assume a typical configuration of a CVM, which uses a standard set of *virtio* devices [50], [51], including *virtio-blk* and *virtio-net* for providing block storage and network connectivity, respectively. Because block devices are controlled by the untrusted host, we assume that the CVM employs a block device integrity protection mechanism, such as *dm-verity* [52], to prevent the host from tampering with the LLM weights stored on disk, which could otherwise be manipulated to facilitate prompt leakage. We also assume that the LLM service and the kernel inside the CVM uses address space layout randomization, ASLR and KASLR, respectively.

Adversarial Capabilities. We adhere to the standard, software attacker model for CVMs, where the attacker controls the

majority of privileged system software including the host VMM, but without physical access to hardware. Specifically, we consider the attacker assumed by Intel TDX [53], since our focus is on LLM services running inside an Intel TDX CVM. The attacker can observe sequences of memory pages and block sectors accessed by the CVMs, through TDX’s page fault side channel and block I/O side channel (see §II-B), respectively. In line with prior work [16], [47], [54], we also assume that the attacker has full offline access to the LLM weights. This captures common deployment scenarios for private LLM services, where the served model is either a publicly available base model or a variant adapted to private downstream tasks via parameter-efficient fine-tuning (PEFT), e.g., LoRA [21]. We do not, however, assume knowledge of the PEFT adapter weights, as these can be securely provisioned to the CVM. When the base model is unknown, the attacker may first attempt model fingerprinting [55] or stealing attacks [17], [56]–[58] via query APIs, side channels, or a combination of both; we consider these directions orthogonal to our focus and leave them to future work (see §IX).

IV. THE SPARSEETY ATTACK

We now present SparSEeTy, a new token extraction attack against sparsity-exploiting LLM serving systems (see §II-A). Our key insight is that these systems selectively access the down projection weights in an LLM’s FFN layers depending on which of its neurons are activated, and that such access patterns can be leaked through side channels and then inverted to recover the tokens processed by the LLM.

Notation & Adversarial Goals. Let $T^{(j)}(\cdot)$ denote the j -th Transformer layer. We define $A^{(j)}(\cdot)$ as the portion of $T^{(j)}(\cdot)$ up to (and including) the activation function within it, whose output is fed into its down projection linear layer denoted by $D^{(j)}(\cdot)$. Since page fault side channels expose activation traces in binary form, we let $B^{(j)}(\cdot)$ denote the binarized neuron activation trace of $A^{(j)}(\cdot)$. We then define $\mathbf{B}(\cdot)$ as the concatenation of these binarized neuron activation traces across all Transformer layers. For an input token x_i , omitting attention to previous tokens for simplicity, this is expressed as:

$$\mathbf{B}(x_i) = B^{(1)}(x_i) \parallel B^{(2)}(T^{(1)}(x_i)) \parallel \dots \quad (1)$$

Using this notation, the sequence of binary activation traces leaked by the victim LLM during inference is given as:

$$\left\{ \mathbf{B}(x_1), \dots, \mathbf{B}(x_n), \mathbf{B}(x_{n+1}), \dots, \mathbf{B}(x_{n+T-1}) \right\} \quad (2)$$

The first n activation traces are leaked during the prefill stage, as the LLM processes the input tokens $(x_1, \dots, x_n)$ to produce the first output token x_{n+1} . The remaining $T - 1$ activation traces are leaked during the decode stage, as the LLM generates subsequent output tokens, starting from x_{n+2} to x_{n+T} .

SparSEeTy’s goals can be formulated as follows: (i) to extract the binary activation trace sequence given in Equation (2) covertly during the victim LLM’s execution, and (ii) given these $n + T - 1$ activation traces, to find a token sequence $(\hat{x}_1, \dots, \hat{x}_{n+T-1})$ that best matches the original sequence $(x_1, \dots, x_{n+T-1})$.

Table I: Comparison between token reconstruction attacks against LLM serving systems.

Attack	Threat model	Leaked internal state	Observation channel	Query access	Weight access	Adversary	Reconstructed tokens
Language Model Inversion [18]	Black-box API	Next-token probabilities	API response	Required	None	API client	System prompt
PIA [16]	Collaborative inference	Intermediate activations (partial layers)	Direct tensor read	None	Required	Malicious participant	Prompt & response
Dong et al. [17]	Collaborative inference	Intermediate activations (partial layers)	Direct tensor read	Required	None	Malicious participant	Prompt & response
I Know What You Said [47]	Co-tenant on shared host	Embedding table lookups	Cache-line access pattern	None	Required	Co-located process	Prompt & response
KV-cache Inversion [54]	Confidential LLM serving	Cached KV values	Direct tensor read	None	Required	Untrusted host	User prompt
TDXRay [31]	Confidential LLM serving	Tokenization hash map lookups	Page/cache-line access pattern	Required	None	Untrusted host	Prompt & response
SparSEEtY (Ours)	Confidential LLM serving	Binary activations	Block/page access pattern	None	Required	Untrusted host	Prompt & response

Attack Flow. SparSEEtY operates in two phases: an online and offline phase. The online phase (see §V) begins with the attacker identifying the GPAs to which the LLM’s down projection weights are loaded (§V-A). The attacker then arms those pages, causing page faults to trigger as the victim LLM processes input prompts and generates output tokens (§V-B). The resulting faults are collected to construct a sequence of binary activation traces for each processed and generated token. To reduce the number of CVM exits and page faults, SparSEEtY also incorporates several optimizations (§V-C).

In the offline phase (§VI), the attacker inverts this sequence of neuron activation traces to reconstruct the original tokens. Starting with the first neuron activation trace to reconstruct the first token, SparSEEtY autoregressively inverts the neuron activation trace of each subsequent token (§VI-A). SparSEEtY uses a search-based method to invert each activation trace (§VI-B) to accurately and efficiently recover the original tokens.

Comparison with Prior Token Reconstruction Attacks. Prior work has investigated token reconstruction for LLMs across a range of settings, as summarized in Table I. These attacks differ along three key dimensions: their threat models, the information leaked and the channels through which it leaks, and the types of tokens targeted for reconstruction.

A line of work proposed to invert the final next-token probabilities of LLMs [18], intermediate activations [16], [17], and KV cache values [54] to reconstruct system or input prompts. These attacks operate, respectively, in a black-box API setting (where the attacker directly observes API responses), a collaborative inference setting (where the attacker observes partial intermediate activations), and a confidential LLM serving setting (where the attacker observes KV cache offloaded to an untrusted host). Another line of work exploits side channels to leak embedding table lookups [47] or tokenization hash map lookups [31] to reconstruct input and output tokens in a co-located process setting and a confidential LLM serving setting, respectively. Our attack assumes a unique setting: the attacker is able to observe (a select subset of) intermediate neuron activations triggered by both prompt and response tokens through a combination of side channels, but, unlike prior work, only at binary resolution; that is, the attacker can determine whether each neuron was activated or not (a binary value), without access to the exact activation magnitude.

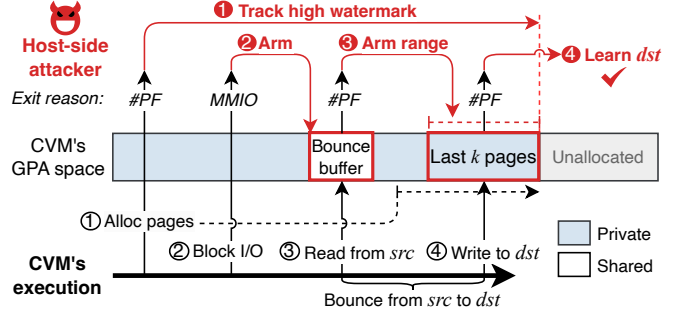


Fig. 2: Finding private GPAs of down projection weights.

Existing attacks assume either query access or offline weight access, but never neither. When offline weight access is not assumed, query access is typically required to profile the victim model’s behavior: Dong et al. use query access to train a surrogate model that mimics the victim LLM [17], and TDXRay uses it to profile the location of hash maps [31]. Our attack assumes offline weight access but not query (inference API) access. We note, however, that, like prior work, query access could substitute for offline weight access by profiling the victim model’s activation patterns over the attacker-chosen token sequences and then matching observed patterns against the profile to reconstruct the victim’s tokens.

V. COLLECTING NEURON ACTIVATION TRACES

The SparSEEtY attack begins with an online phase, which takes place while the victim LLM serving system is processing tokens (hence the term “online”). During this phase, the attacker observes the victim system via a combination of block and page access oracles, and extracts a neuron activation trace for each token processed by the LLM. SparSEEtY extracts these traces by monitoring selective access patterns to the down projection weights of FFN neurons, arising from sparse activations followed by sparse down projection.

A. Finding GPAs of Down Projection Weights

The first step in collecting neuron activation traces during LLM inference is to identify the GPAs of down projection weights selectively accessed due to *sparse* down projection. To this

end, we propose to combine block access oracle with page allocation and access oracles, as detailed below.

Fig. 2 illustrates how we combine them to learn the GPAs of down projection weights. During bootstrapping, the victim LLM system inside the CVM maps the model file into its address space, prompting the OS’s block layer to allocate CVM-private pages to receive block data (see ①). When the CVM accesses previously unallocated pages, it triggers an SEPT page fault, which the attacker uses to track the high watermark of the CVM’s GPA space (①).

Next, the CVM’s *virtio-blk* driver issues block read requests (②). SparSEETy monitors the requests that target the down projection weights, and learns the GPA of the bounce buffer to which these weights are copied. Selective weight accesses during LLM inference, however, do not occur on the bounce buffers; instead, they occur on the CVM-private pages within the CVM’s OS page cache, where the weights are ultimately copied. To this end, SparSEETy arms the bounce buffer (②) and resumes the CVM.

Once resumed and signaled of block I/O completion, the CVM reads the bounce buffer (denoted *src*) for bouncing (③), triggering an EPT fault. Upon receiving this fault, SparSEETy disarms the bounce buffer, and arms the last k pages below the high watermark (③), one of which is the final destination (*dst*) of the bounced data. While SparSEETy could block all guest pages by setting k to the total number of guest pages, doing so would incur excessive TDH.MEM.RANGE.BLOCK SEAMCALLs. For finding the GPAs of down-projection weights, we set $k = 16,000$ based on our empirical observation that the LLM’s large size causes it to be loaded into pages near the high watermark.

SparSEETy then resumes the CVM, which immediately triggers another CVM exit with an SEPT fault, as the CVM attempts to write the bounced data to an OS page-cache page (④). By examining the GPA of the faulting page, the attacker finally learns the GPAs where the down projection weights are ultimately stored in CVM-private memory (④).

B. Extracting Activations via Page Faults

Having identified the GPAs of the CVM-private pages containing the down projection weights in the previous step, we can now determine which neurons are activated in each FFN layer by inducing and monitoring page faults when the CVM reads these weights during sparse down projection.

Observability of Per-Neuron Activation. An inherent limitation of controlled-channel attacks is that it reveals memory access patterns only at the granularity of pages. In practice, however, this does not hinder SparSEETy’s ability to monitor neuron activations. In modern LLMs, the hidden dimension size (i.e., d_{model}), which determines the size of the down projection weights for a single neuron, typically reaches into the thousands, with each weight stored using 16-bit floating-point precision. As a result, the down projection weights for a single neuron typically span an entire 4KiB page (or more) in memory. For example, in Llama-2-7b [59], the size of the down projection weights per neuron is 8KiB. By monitoring either of the two

4KiB pages, SparSEETy can reliably determine whether each neuron was activated.

Handling Model Parallelism. Modern LLM systems employ model parallelism [45], where the down projection $D^{(j)}(\cdot)$ is split across multiple threads, which run across different virtual CPU (vCPU) cores in the case of CVMs. Each thread simultaneously processes a subset of neurons during the down projection, and the partial results of projection are then reduced (i.e., summed) to produce the full matrix-vector multiplication result. SparSEETy handles this model parallelism, by collecting concurrent page faults from different vCPUs and aggregating them for each token.

Handling Batched Prefill Computation. During the prefill stage, LLM systems typically process the entire prompt $(x_1, \dots, x_n)$ as a batch, sending all tokens through each layer before moving on to the next. For $D^{(j)}(\cdot)$, this means that the system computes $D^{(j)}(x_1), \dots, D^{(j)}(x_n)$ sequentially before $D^{(j+1)}(\cdot)$. SparSEETy handles this batched computation by disentangling the per-token activation patterns within each layer, and then aggregating these layer-wise patterns across all Transformer layers to construct complete activation profiles for each position in the prompt token sequence.

C. Optimizations for Enhancing Stealthiness

Using page faults for monitoring neuron activations could introduce significant overhead due to VM exits triggered by page faults. This slows down the CVM’s LLM inference, raising the likelihood of detection. We therefore propose two optimizations to enhance SparSEETy’s stealthiness.

Selective Neuron Monitoring. One could monitor every neuron in every FFN layer collecting the full binary activation trace $B(x_i)$ for each token x_i , but this requires installing a prohibitively large number of page fault probes, resulting in a page fault for every activated neuron. To mitigate this, we propose a selective neuron monitoring strategy.

- **Layer Selection:** We first select activations of the lower layers of the victim LLM for monitoring, based on the finding that the activations of lower layers that are closer to the token embedding layer contain more direct information about the input [15]. SparSEETy uses the activation trace of the first FFN layer for each token, i.e., $B^{(1)}(x_i)$ for x_i .
- **Neuron Selection:** Within the first layer, we further select a subset of neurons whose combined activations maximize both information content and discriminative power [14], as follows. Using a public dataset $\mathcal{A}$, we generate binary activation traces $B^{(1)}(x)$ for all tokens $x \in \mathcal{A}$, forming the empirical distribution used to compute entropy. We begin by selecting the neuron with the highest individual entropy, then iteratively expand the set via a greedy search, each time adding the neuron that yields the largest increase in joint entropy. This yields a compact yet informative subset of neurons capable of distinguishing diverse activation patterns. We denote the partial activation trace of this selected subset in the k th layer by $\tilde{B}^{(k)}(\cdot)$;

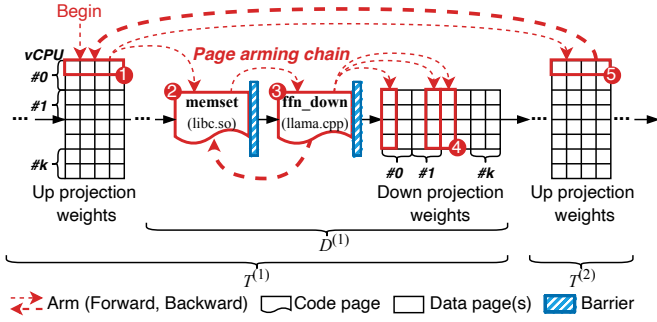


Fig. 3: Page-arming chain used to monitor the activations of the first FFN layer under multi-threaded, model-parallel LLM inference.

during inference, SparSEety monitors $\tilde{B}^{(1)}(x_i)$ for each token x_i .

This two-stage neuron selection process of SparSEety yields the following pruned sequence of activations when observing the token sequence $(x_1, \dots, x_{n+T-1})$:

$$\left\{ \tilde{B}^{(1)}(x_1), \dots, \tilde{B}^{(1)}(x_n), \tilde{B}^{(1)}(x_{n+1}), \dots, \tilde{B}^{(1)}(x_{n+T-1}) \right\} \quad (3)$$

We show later that monitoring this pruned set of neurons is sufficient for reconstructing the tokens with high accuracy.

Single-Step-Free, Chained Page Arming. A naive approach to monitoring memory accesses via page faults is to arm each monitored page, single-step the CVM on every VM exit caused by accessing an armed page, and re-arm the page upon another VM exit caused by single-stepping. While using this approach ensures that every access to the down projection weights is monitored, it has two critical drawbacks: (i) it incurs significant overhead due to a large number of VM exits, and (ii) modern CVMs implement mitigations that can significantly slow down single-stepping attacks [30], [60].

We therefore propose chained page arming, a single-step-free approach tailored to monitoring accesses to down projection weights from multiple vCPUs during multi-threaded, model-parallel LLM inference. Fig. 3 depicts the chain, consisting of five types of CVM-private pages: ① an up projection weight page of $T^{(1)}$, ② a code page that contains the definition of `memset` invoked by each thread to zero an intermediate buffer during $D^{(1)}$, ③ a code page that contains the definition of the sparse down projection function executed by each thread, ④ the selected down projection weight pages of $T^{(1)}$ being monitored, and ⑤ an up projection weight page of $T^{(2)}$.

To monitor accesses to these pages, SparSEety first identifies their GPAs using the method described in §V-A. SparSEety then begins monitoring LLM inference on prompt $(x_1, \dots, x_n)$ by arming ①. When a page fault occurs on ①, SparSEety disarms it and arms ② and ⑤. This continues along the chain, with each fault disarming the faulting page and arming the next page(s) in the chain. The disarmed pages must be armed again to monitor page accesses created by tokens subsequently processed. To this end, we introduce two backward edges: (i) a

Algorithm 1 SparSEety’s activation trace inversion.

```

1: Input: Activation traces  $\{\tilde{B}^{(1)}(x_i)\}_{i=1}^{n+T-1}$ , victim LLM, vocabulary  $\mathcal{V}$ 
2: Output: Reconstructed token sequence  $(\hat{x}_1, \dots, \hat{x}_{n+T-1})$ 
3: for  $i = 1$  to  $n + T - 1$  do ▷ Loop for autoregressive reconstruction
4:   if  $i = 1$  then
5:      $\mathbf{P}_i \leftarrow \text{Uniform}(\mathcal{V})$  ▷ No prior context
6:   else
7:      $\mathbf{P}_i \leftarrow \{P(v \mid \hat{x}_1, \dots, \hat{x}_{i-1}) : v \in \mathcal{V}\}$  ▷ Full forward pass
8:   end if
9:    $(v_1, v_2, \dots, v_{|\mathcal{V}|}) \leftarrow \text{Sort } \mathcal{V} \text{ by } \mathbf{P}_i \text{ (descending)}$ 
10:  for  $j = 1$  to  $|\mathcal{V}|$  do
11:     $B_{v_j} \leftarrow \tilde{B}^{(1)}(v_j)$  ▷ First-layer forward pass
12:    if  $B_{v_j} = \tilde{B}^{(1)}(x_i)$  then
13:       $\hat{x}_i \leftarrow v_j$ 
14:      break ▷ Stop at the first match
15:    end if
16:  end for
17: end for
18: return  $(\hat{x}_1, \dots, \hat{x}_{n+T-1})$ 

```

fault triggered at ③ arms ②, which allows SparSEety to monitor page accesses created by the next prompt token processed in the batched down projection loop, $D^{(1)}(x_1), \dots, D^{(1)}(x_n)$, and (ii) a page fault at ⑤ arms ①, allowing SparSEety to monitor page accesses created by the next response token in the autoregressive generation loop, $T^{(1)}(x_n), \dots, T^{(1)}(x_{n+T-1})$.

The two code pages ② and ③ executed by all threads serve as double-turnstile barriers [61] during model-parallel execution of $D^{(1)}(x_1, \dots, x_n)$. The page faults at these pages demarcate token boundaries in each vCPU, and the barriers force all vCPUs to synchronize before proceeding to the next prompt token. This enables SparSEety to aggregate partial access patterns of $D^{(1)}(x_i)$ from all vCPUs for each prompt token x_i . Even when a vCPU does not access any monitored down projection weights for x_i , the barrier-induced page faults still mark when $D^{(1)}(x_i)$ begins and ends on that vCPU.

VI. INVERTING NEURON ACTIVATION TRACES

SparSEety then proceeds to this offline phase, where the neuron activation traces collected in the earlier online phase are inverted to reconstruct the original tokens.

A. Autoregressive Trace Inversion

We reconstruct the sequence $(\hat{x}_1, \dots, \hat{x}_{n+T-1})$ autoregressively starting from the first token $\hat{x}_1$. Specifically, we invert $\tilde{B}^{(1)}(x_1)$ to reconstruct $\hat{x}_1$, $\tilde{B}^{(1)}(x_2)$ to reconstruct $\hat{x}_2$, and so on. At iteration i , we condition the reconstruction of $\hat{x}_i$ on all previously reconstructed tokens $(\hat{x}_1, \dots, \hat{x}_{i-1})$. We adopt this greedy, autoregressive approach, because the alternative, which treats all tokens as simultaneous variables and searches for a global optimum that elicits the same neuron activations, is computationally infeasible. With the large vocabularies of modern LLMs, the time complexity of such a global search grows exponentially with the length of the token sequence.

A possible concern with the greedy approach is that an error in reconstructing one token could propagate and degrade the accuracy of subsequent token reconstruction. In practice, however, we observe that such error propagation is limited (see §VIII-B). SparSEETy’s reconstruction relies on activations from the first layer of the model, which preserve direct information about the input token before extensive contextualization occurs in deeper layers [62].

B. Search-Based Trace Inversion

At i th iteration of the autoregressive sequence reconstruction, SparSEETy searches for the token whose activation matches the observed activation trace $\tilde{B}^{(1)}(x_i)$.

A Strawman Approach: Exhaustive Vocabulary Search. A straightforward approach is to compare $\tilde{B}^{(1)}(x_i)$ with the activation patterns of all possible tokens in the vocabulary. That is, it performs a forward pass of the first layer of the victim LLM for every token in the vocabulary $\mathcal{V}$. This yields $|\mathcal{V}|$ activation traces, which we then binarize and prune to obtain $\{\tilde{B}^{(1)}(v) \mid v \in \mathcal{V}\}$. After this partial forward pass for each v , we compare its activation trace $\tilde{B}^{(1)}(v)$ with the observed activation trace $\tilde{B}^{(1)}(x_i)$. If they match, v is flagged as a candidate for successful reconstruction. This approach suffers from two limitations, however: (i) multiple candidate tokens may produce identical binary activation patterns, rendering the search ambiguous, and (ii) the time complexity is prohibitively high for LLMs with a large $|\mathcal{V}|$.

Our Approach: Guided Vocabulary Search. To resolve activation ambiguities, we exploit the target LLM’s inherent next-token probability distribution over its vocabulary. This distribution, calculated for the i th position conditioned on the previously recovered token sequence $(\hat{x}_1, \dots, \hat{x}_{i-1})$, naturally ranks all possible tokens by their contextual likelihood. We examine candidates in descending probability order, terminating when a candidate’s activation pattern matches the observed trace. As we show in §VIII, this prioritization achieves higher accuracy in the presence of pattern ambiguity, compared to an unguided, exhaustive search.

A potential drawback of this approach over the strawman one is that it additionally requires a full forward pass when reconstructing each token. However, we observe that matching activations are often found within a few high-probability next tokens produced by the full forward pass. This allows the search to terminate earlier than the naive method, effectively limiting the impact on inversion speed (see §VIII-D).

Alternative: Optimization-Based Trace Inversion. We also considered an optimization-based approach, with the optimization goal of minimizing the disparity between the generated activation and the target activation trace. The inversion can be solved by initializing the embedding vector randomly, computing a loss based on the difference between the generated and target traces, and then backpropagating that loss to update the initialized embedding vector. The final updated embedding vector after optimization would closely approximate the original token’s embedding. However, we ultimately rejected this

approach because the optimization process yielded a weak loss signal. Since the target activation trace is only available in its binarized form in our setting (i.e., a sequence of 0s and 1s, rather than continuous real values), the resulting objective function was too sparse to reliably guide the gradient-based optimization toward the correct embedding.

VII. IMPLEMENTATION DETAILS

We implemented SparSEETy atop Linux Kernel Virtual Machine (KVM) [19] (v6.8.0) and QEMU [20] (v8.2.50).

Identifying Logical Block Addresses. To monitor the CVM’s accesses to blocks that are loaded into the guest OS page cache and subsequently monitored by SparSEETy (i.e., the nodes in the page arming chain), we compute the logical block address (LBA) of each target block as follows. First, we read the guest disk image’s partition table, identify the relevant partition, and retrieve the file’s inode. From this inode, we derive the file’s block offset, and add the partition’s starting LBA to obtain the file’s starting LBA. We then convert the byte offset of the target block within the file into a block offset, and add the file’s starting LBA to obtain the target block’s LBA.

Finding GPAs of Loaded Blocks. We modified QEMU’s *virtio-blk* device to intercept the CVM’s accesses to target blocks. Upon interception, it invokes SparSEETy’s kernel module to request arming of the bounce buffer page, and the last k private pages below the high watermark upon bounce buffer faults. The module exposes an *ioctl* interface for arming pages and retrieving the faulting page’s address, allowing SparSEETy to learn the private GPA of each loaded block.

Chained Page Arming. Once the private GPAs of all nodes in the chain are known, SparSEETy activates the chain by sending these GPAs to its kernel module. To efficiently track varying numbers of monitored down projection weight pages, SparSEETy maintains a per-CVM red-black tree of their GPAs. Chained page arming is implemented primarily within the kernel module, by hooking the page fault handler to disarm the faulting page and arm the next page(s) in the chain. We track the armed state of each monitored page, and use a per-page lock to ensure that only a single vCPU arms or disarms a page, even when multiple vCPUs fault on it concurrently. The handler exits to the user-mode VMM (i.e., QEMU) where SparSEETy logs the accessed down projection weights and token changes. These logs are then used in the offline phase and inverted to reconstruct the tokens.

VIII. EVALUATION

Target Models. We used OPT-6.7b [63], ReluLLaMA-7B [64], Nemotron-3-8B-Base-4k [65], Llama-2-7b [66], and Gemma-7b [67] for our evaluation, with varying properties, as summarized in Table II. Notably, these models use different activation functions: ReLU, ReLU², SwiGLU, and GeGLU. For Llama-2-7b and Gemma-7b that use SwiGLU and GeGLU respectively, we applied magnitude-based thresholding that prunes the lowest 50% of activations by magnitude in each layer, following Federici et al. [25]. The models have different numbers of

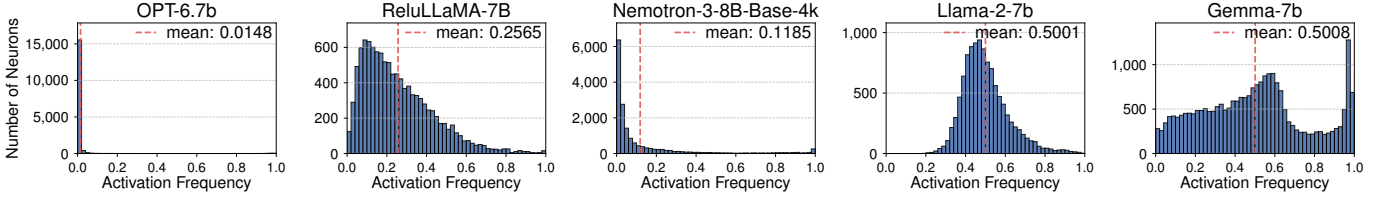


Fig. 4: Distribution of activation frequencies (fraction of tokens activating each neuron) on the Wikipedia dataset. Red dashed lines indicate average frequencies.

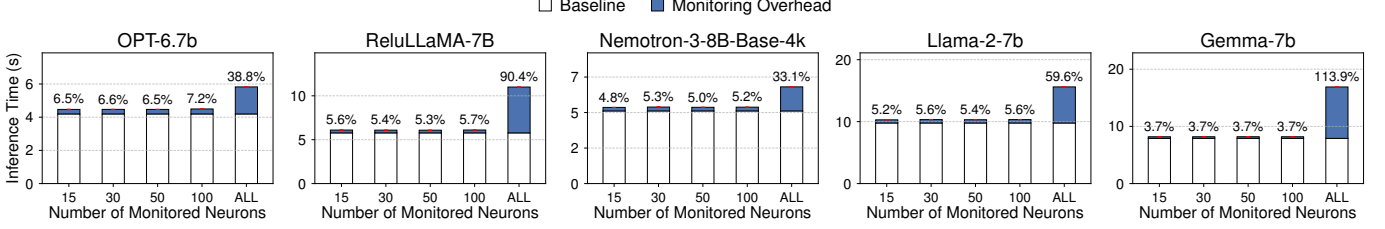


Fig. 5: Wall-clock inference time before & after SparSEety’s monitoring. Results were obtained while each LLM was processing a prompt in Skytrax Reviews, averaged over 100 trials. The “ALL” column shows the overhead without selective monitoring.

Table II: LLMs targeted in our evaluation.

Model	Activation	Layers (N)	Hidden Dim (d_{model})	FFN Dim (d_{ff})	Neuron Size*	Vocab Size ($ V $)
OPT-6.7b	ReLU	32	4,096	16,384	8,192	50,272
ReluLLaMA-7B	ReLU [†]	32	4,096	11,008	8,192	32,000
Nemotron-3-8B-Base-4k	ReLU [‡]	32	4,096	16,384	8,192	256,000
Llama-2-7b	SwiGLU [‡]	32	4,096	11,008	8,192	32,000
Gemma-7b	GeGLU [‡]	28	3,072	24,576	6,144	256,000

* The size of each neuron is measured in bytes, and calculated as $d_{\text{model}} \times 2$ bytes.

[†] This model was ReLU-fied, after training.

[‡] We applied magnitude-based thresholding.

neurons in each layer and different vocabulary sizes, while the size of each neuron is 8KiB in most models and 6KiB in Gemma-7b. The models also exhibit different activation sparsity levels and patterns, per our own profiling shown in Fig. 4. ReLU-variant models show skewed distribution with varying sparsity levels, whereas Llama-2-7b and Gemma-7b after pruning show much denser activation patterns, with mean frequencies near 0.5.

Datasets. We used five datasets: three real-world datasets—Skytrax Reviews [68], Medical WiKiDoc [69], and ECHR Law [70]—and two synthetic datasets—Private Prompts [71] and System Prompt Leakage [72]. These datasets contain sensitive prompts that elicit sensitive responses, and have therefore been used extensively in prior token leakage attacks [18], [73]–[75]. Unless stated otherwise, we randomly sampled 100 prompts from each dataset for evaluation. To train the LoRA adapters, we used the LaMP-4 [76], a dataset widely used in prior work on personalized generation [77]–[79].

A. Inference-Time Overhead

We first quantify the inference-time overhead introduced by SparSEety’s neuron activation monitoring during the online phase, i.e., during the victim LLM service’s inference.

Experimental Setup. We targeted PowerInfer [7], an optimized LLM serving system built on llama.cpp [80] that uses sparse down projection based on activation sparsity. We incorporated magnitude-based thresholding into PowerInfer to induce activation sparsity for SwiGLU and GeGLU models. While PowerInfer also supports sparse up projection using an activation predictor, we disabled this feature, because (i) it degrades inference accuracy and (ii) predictors are not available for certain models that we used in our evaluation. We deployed our modified PowerInfer on a server equipped with dual Intel Xeon Gold 6548Y+ processors (128 logical cores in total) and 1 TiB of RAM. We configured PowerInfer to use 16 threads, and ran it inside a 16-vCPU, 32GB-RAM CVM so as to enable different forms of parallelism during inference. The vCPUs were pinned to physical CPU cores to ensure stable performance measurements. We used the default configuration of PowerInfer otherwise. We used a single dataset—Skytrax Reviews—for measuring and analyzing inference-time overhead, since our goal here is to quantify the overhead introduced by SparSEety’s neuron activation monitoring, which is largely independent of the input prompts.

Wall-Clock Inference Time. We first measure the wall-clock inference time, i.e., the time each LLM takes to consume a single prompt and generate 30 tokens, with and without SparSEety’s neuron activation monitoring. Fig. 5 presents our measurements. The overhead ranges from 3.7% to 7.2% depending on the model, when monitoring 15 to 100 neurons. When monitoring 100 neurons, which was sufficient for SparSEety to achieve up to 100% reconstruction accuracy across all five models (see §VIII-B), the overhead remains at this level. The overhead increases up to 33%–114% when monitoring all first-layer FFN neurons (denoted by ALL). The results show that our selective neuron monitoring (§V-C) effectively reduces

Table III: The number of per-token page faults averaged over Skytrax Reviews. The ALL column monitors every neuron in the first FFN layer and serves as the baseline; percentages in parentheses are relative to this baseline.

Model	Number of Monitored Neurons				
	15	30	50	100	ALL
OPT-6.7b	46.0 $\pm$ 0.2 (17.97%)	53.4 $\pm$ 0.0 (20.86%)	61.9 $\pm$ 0.3 (24.15%)	81.6 $\pm$ 0.3 (31.86%)	256.1 $\pm$ 1.6
ReluLLaMA-7B	43.4 $\pm$ 0.1 (0.63%)	55.8 $\pm$ 0.2 (0.81%)	71.1 $\pm$ 0.1 (1.04%)	107.5 $\pm$ 0.1 (1.57%)	6,848.4 $\pm$ 11.1
Nemotron-3.8B-Base-4k	41.2 $\pm$ 0.0 (10.08%)	43.9 $\pm$ 0.7 (10.75%)	46.0 $\pm$ 0.3 (11.25%)	49.4 $\pm$ 0.1 (12.08%)	408.9 $\pm$ 2.8
Llama-2-7b	41.7 $\pm$ 0.3 (0.75%)	49.1 $\pm$ 0.9 (0.89%)	58.2 $\pm$ 0.7 (1.05%)	82.3 $\pm$ 1.3 (1.49%)	5,539.2 $\pm$ 1.6
Gemma-7b	41.2 $\pm$ 0.0 (0.33%)	48.9 $\pm$ 0.1 (0.40%)	58.7 $\pm$ 0.1 (0.48%)	81.7 $\pm$ 0.4 (0.66%)	12,320.3 $\pm$ 3.2

Table IV: Breakdown of total CPU time during OPT-6.7b’s inference on a Skytrax Reviews prompt, shown for the runs with the minimum, median, and maximum total CPU time across 10 trials.

Time spent by ...	Min-time run	Median-time run	Max-time run
Guest vCPU	69,918 (96.05%)	70,368 (95.59%)	70,032 (94.61%)
Host User-Mode	507 (0.70%)	713 (0.97%)	887 (1.20%)
Host Kernel-Mode	2,367 (3.25%)	2,535 (3.44%)	3,100 (4.19%)
- Barrier Wait	581 (0.80%)	762 (1.04%)	1,028 (1.39%)
- IPI Calls	9 (0.01%)	9 (0.01%)	8 (0.01%)
- SEAMCALLs	27 (0.04%)	26 (0.04%)	26 (0.03%)
- Others	1,751 (2.41%)	1,738 (2.36%)	2,038 (2.75%)
Total CPU Time	72,792 (100.00%)	73,615 (100.00%)	74,020 (100.00%)

inference-time overhead, improving stealthiness.

Number of Per-Token Page Faults. We now measure the number of page faults triggered during LLM inference while monitoring varying numbers of neurons. We report per-token page fault counts, i.e., the number of page faults triggered while processing each token, because the same prompt can be tokenized into different lengths depending on the model (e.g., 64 tokens for ReluLLaMA and Llama, vs. 56 tokens for OPT, Nemotron, and Gemma). Table III presents the results. As expected, the per-token page fault count rises as more neurons are monitored. When monitoring all neurons in the first-layer FFN, the number of page faults increases substantially for ReluLLaMA-7B, Llama-2-7b, and Gemma-7b, which are less sparse than the other two models (see Fig. 4). These results demonstrate that our selective neuron monitoring substantially reduces page faults and thus inference-time overhead.

Overhead Breakdown. We further analyze the sources of inference-time overhead, when SparSEETy monitors 100 neurons during OPT-6.7b’s inference with 16 vCPUs on a Skytrax Reviews prompt. We measure per-core CPU times by instrumenting several kernel functions with Tracepoint [81], and attaching eBPF [82] programs to compute the time elapsed between them. Table IV shows the breakdown of the total CPU time for three runs: those with the minimum, median, and maximum total CPU time. The dominant off-VM overhead stems from barrier synchronization used to demarcate prompt tokens during batched prefill. The contribution of IPI calls and SEAMCALLs is not significant: the overhead of IPI calls

for TLB shutdowns and of arming and disarming pages is small compared to the total barrier synchronization overhead for long prompts. However, as the number of monitored neurons increases, we expect their overhead to become more pronounced. Finally, the impact on vCPU execution time due to cache pollution or TLB shutdowns appears negligible, likely due to the memory-bound nature of LLM inference.

B. Reconstruction Accuracy

We now evaluate the reconstruction accuracy of SparSEETy’s inversion during the offline phase.

Experimental Setup. We inverted the collected traces using an NVIDIA H100 GPU with a 94GB VRAM. The software environment was PyTorch 2.7.1 (built with CUDA 12.4) with NVIDIA driver 550.67. To evaluate the reconstruction accuracy of our inversion methods, we use BLEU [83], which assesses the n -gram overlap between the original and reconstructed token sequences, effectively indicating the success rate of token recovery. We used the Wikipedia dataset [84] with approximately 135,000 tokens to compute joint entropy used in neuron selection (see §V-C). We begin with the top 15 neurons ranked by joint entropy, which represents the theoretical minimum required to uniquely identify all 32,000 ($\approx 2^{15}$) tokens in the vocabulary. This minimum requirement corresponds to the smallest vocabulary size among the models we evaluated, ReluLLaMA-7B and Llama2-7b. We then gradually increase the number of selected neurons up to 100.

Quantitative Results. Fig. 6 presents the reconstruction accuracy across all target models and datasets. With only 100 monitored neurons, SparSEETy consistently achieves near-100% reconstruction accuracy across all models and datasets that were evaluated. Notably, OPT-6.7b, which exhibits the highest activation sparsity, requires 80 or more neurons to approach this level, whereas other models with lower sparsity achieve comparable accuracy with only 30 neurons. This indicates that reconstructing highly sparse activations requires monitoring more neurons. We also find that vocabulary size has limited influence on our search-based reconstruction. Although the vocabulary of Nemotron and Gemma-7b is an order of magnitude larger than that of other models, their reconstruction performance remains comparable, suggesting that activation patterns across tokens are sufficiently distinct.

Qualitative Examples. We showcase several reconstructed prompts and responses in Table V. As shown, SparSEETy accurately reconstructs not only prompt tokens but also response tokens, and recovers both common filler words (e.g., “a”, “the”, “is”) and highly specific tokens such as domain-specific terms and proper nouns (e.g., “MSWG”). Moreover, SparSEETy maintains high accuracy even for long sequences containing many tokens. Finally, SparSEETy successfully reconstructs subsequent tokens even after a reconstruction error, validating the limited error propagation discussed in §VI-A.

C. Robustness to Fine-tuning & Partial CPU Offloading

We evaluate our attack under two additional deployment scenarios that restrict the attacker’s visibility: (i) the base

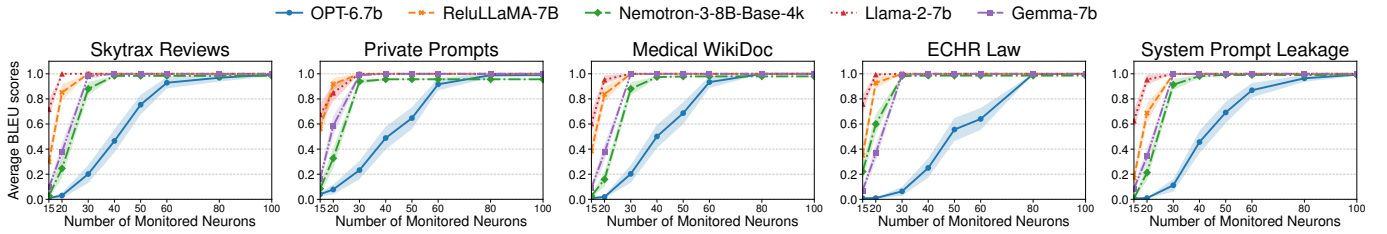
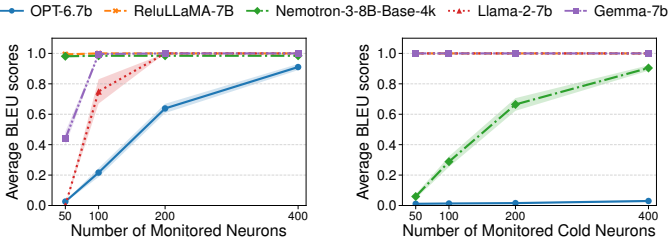


Fig. 6: Reconstruction accuracy across five datasets, with varying numbers of monitored neurons. Measured by BLEU scores.

Table V: Examples of tokens reconstructed by inverting activation traces from 15 to 40 first-layer FFN neurons across five target LLMs. All prompts are from the Private Prompts dataset to illustrate the privacy risk.

Target Model	# of Neurons	Original Prompt & Response	Reconstructed Prompt & Response
OPT-6.7b	40	What is a question which results in the answer of “19 April 1984”? Answer: The date of the first ever Test match between India and Pakistan.	What is a question which results in the answer of “19 April <u>2001</u> ”? Answer: The date of the first ever Test match between India and Pakistan.
ReluLLaMA-7B	30	Please add punctuation to this: Latest visit November 2227.\n Latest visit: November 22, 2027.	Please add punctuation to this: Latest visit <u>September</u> 2227.\n Latest visit: November 22, 2027.
Nemotron-3-8B-Base-4k	20	If Sarah Smith needs to buy 3 loaves of bread at the store, and each loaf costs \$2, how much money will Sally spend in total?\n Answer: 3 loaves of bread * \$2 each = \$6	If Sarah Smith needs to buy 3 loaves of bread at the store, and each loaf costs \$2, how much money will Sally spend in total?\n Answer: 3 loaves of bread * \$2 each = \$<u>amp</u>
Llama-2-7b	15	what was the number one song on 17 April 1970??\n The number one song on 17 April 1970 was “The Candy Man” by Sammy Davis Jr.	what was the number one song on 17 April 1970??\n The number one song on 17 April 1970 was “The Candy Man” by Sammy Davis Jr.
Gemma-7b	30	Write an email with the following subject: Joint MSWG & S&PWG meeting, January 2011.\n\n The email should be sent to the following email addresses:\n\n * s&pwg-chair@w3	Write an email with the following subject: Joint MSWG & S&PWG meeting, January 2011.\n\n The email should be sent to the following email addresses:\n\n * s&pwg-chair@w3



(a) After adapting models with private LoRA adapters. (b) After offloading cold neuron computation.

Fig. 7: Reconstruction accuracy obtained after (a) adapting the base models with private LoRA adapters with rank 8, and (b) offloading cold neuron computation to the CPU, measured by BLEU scores on Skytrax Reviews.

model is fine-tuned with private LoRA adapters unknown to the attacker, and (ii) only infrequently activated (i.e., *cold*) neurons are offloaded to the CPU, leaving the rest *hot* neurons outside the attacker’s observation scope.

Experimental Setup. To evaluate the effectiveness of SparSEety on LoRA-adapted models, we replicated the LoRA fine-tuning setup for personalized news headline generation from Tan et al. [77], in which rank-8 adapters are applied to the attention layers and trained for two epochs on the LaMP-4 dataset [76], with a learning rate of $1e-5$ and a batch size of 8. We confirmed successful fine-tuning by reproducing the final performance numbers reported in the original study [77],

and examined SparSEety’s token reconstruction performance without access to the adapters.

In the CPU offloading experiment, we first profiled neuron activations using the Wikipedia dataset [84] to identify cold neurons. Assuming that GPU memory can accommodate half of the model parameters, we offloaded the 50% least frequently activated neurons in each layer to the CPU. We then recomputed the joint entropy over these offloaded cold neurons on the same Wikipedia dataset to select which ones to monitor.

Reconstruction Accuracy with LoRA Fine-tuning. We collected neuron activation traces of each model fine-tuned with a private LoRA adapter through SparSEety’s neuron activation oracle, and then reconstructed the tokens that produced these activations with its base model through our search-based, autoregressive inversion (see §VI). As shown in Fig. 7a, BLEU scores improve as more first-layer neurons are monitored, reaching up to 1.0000. All models except OPT-6.7b reach a BLEU score of 0.9845 or higher with 200 monitored neurons; OPT-6.7b reaches 0.9103 with 400 neurons. This robustness suggests that fine-tuning typically alters only a limited subset of model parameters, largely preserving (i) first-layer activation patterns, and (ii) next-token probability rankings.

Reconstruction Accuracy with Partial CPU Offloading. Fig. 7b shows reconstruction accuracy when the attacker observes cold neuron activations. As expected, this setup is more challenging than monitoring all neurons. Even with 400 monitored neurons, Nemotron-3-8B-Base-4k reaches a BLEU score of 0.9040, and OPT-6.7b performs significantly worse, reaching only 0.0296. This means that, in these models,

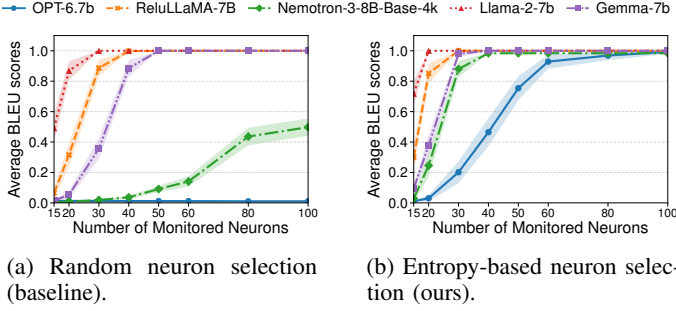


Fig. 8: Reconstruction accuracy obtained via (a) random neuron selection and (b) SparSEety’s entropy-based neuron selection. Measured by BLEU scores on Skytrax Reviews.

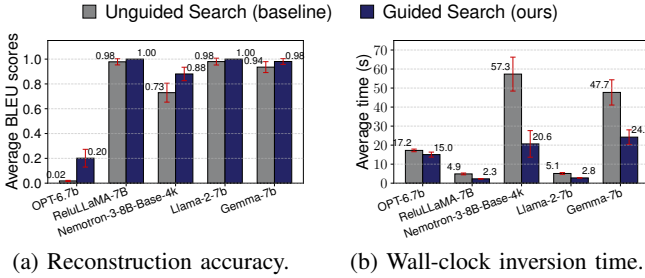


Fig. 9: Effectiveness of SparSEety’s guided search in terms of (a) reconstruction accuracy and (b) computation time. Results were obtained by inverting binary activation traces of top-30 high-entropy neurons on Skytrax Reviews.

neurons with high joint entropy (i.e., discriminative w.r.t. the input tokens) coincide with hot neurons, which are no longer observable to the attacker. For the rest of three models, however, cold neurons are sufficiently discriminative, reaching a BLEU score of 1 with only 50 neurons.

D. Ablation Studies

Random vs. Entropy-based Neuron Selection. We evaluate our entropy-based neuron selection technique by comparing it against random selection and analyzing how reconstruction accuracy differs between the two. Fig. 8 depicts their reconstruction accuracy measured in BLEU scores. All models show improved performance over random selection, particularly OPT-6.7b and Nemotron-3-8B-Base-4k. For Llama-2-7b, only 20 neurons already yield a BLEU score of 1.0, and all models except OPT-6.7b achieve satisfactory reconstruction with as few as 30 neurons. The OPT-6.7b model requires more neurons, likely due to its highly sparse activations. Nevertheless, 100 neurons constitute only 0.61% of the 16,384 neurons in the first layer of OPT-6.7b. Considering that random selection for OPT-6.7b did not achieve a BLEU score of 0.1 even with 100 neurons, our entropy-based selection yields a substantial improvement in reconstruction performance.

Unguided vs. Guided Search. Search-based activation inversion suffers from multiple matches: multiple tokens can produce

identical activation pattern once binarized. This ambiguity increases when only a subset of neurons is monitored. SparSEety addresses this by prioritizing candidate tokens according to their logits (see §VI-B). To validate this strategy, we conducted experiments that compare the BLEU scores obtained when monitoring 30 neurons, using either the vocabulary’s token ID order (denoted by Unguided Search) or our logit-based ordering (Guided Search). The results presented in Fig. 9a demonstrate that our guided search consistently improves reconstruction accuracy.

We also evaluate how quickly activation traces extracted from the victim can be inverted. Fig. 9b compares the wall-clock time required to invert activation traces using unguided and guided search. Although guided search requires a full forward pass to compute next-token probabilities, it achieves faster inversion by terminating the search earlier. Since inversion can be performed offline, our attack’s success is largely insensitive to inversion speed. We nevertheless report these results (i) to demonstrate that inversion can be completed within a practical timeframe, and (ii) to support future research on real-time or dynamically-adapting inversion attacks, whose covert operation may benefit from faster inversion.

IX. DISCUSSION & LIMITATIONS

Potential Mitigations. A straightforward mitigation against our attack is to disable sparsity-exploiting optimizations altogether in LLM serving systems. However, doing so forfeits their substantial efficiency gains (e.g., up to $2\times$ inference speedup [4]).

A more lightweight alternative is to randomize the GPAs of neurons, preventing the attacker from inferring neuron activations through page-access monitoring. This can be achieved by randomizing the placement of the down projection weights within the model file, and keeping the file (or the entire block device) encrypted at rest on the host side. Block encryption alone, e.g., via *dm-crypt* [85], is insufficient, since it preserves the block-level layout of the model. Such randomization is also static—the layout does not change across requests—so an attacker with query access to the victim (which we do not assume) can still profile neuron activation patterns using chosen prompts. A more principled, albeit more expensive, defense is to dynamically reshuffle access patterns via oblivious RAM techniques [86]–[88].

One might also defeat our attack by executing LLM inference entirely on GPUs. However, when a model exhibits high sparsity (e.g., fewer than 10% of neurons active, as in OPT models), it is often more efficient to perform sparse matrix-vector multiplication on the CPU. Indeed, many LLM-serving systems already offload sparse computation to the CPU to better saturate CPU/GPU compute and reduce GPU I/O bandwidth and memory demands [8], [89]–[91]. Our evaluation shows that even when only 50% of the FFN computation runs on the CPU, our attack still achieves high reconstruction accuracy across most models. Finally, recent work by Chrapek et al. shows that even fully CPU-based LLM inference inside CPU TEEs can be more cost-efficient than GPU TEEs such as those of NVIDIA’s H100 [44].

Another mitigation is to keep the model weights private, since our attack assumes the adversary has weight access. However, this defense can often be circumvented. If the victim uses an open-source model but the attacker lacks knowledge of the model, our attack can be combined with prior techniques that infer model hyperparameters and architecture [56]–[58], [92], enabling the attacker to fingerprint and retrieve the corresponding model from a public repository. For proprietary models, previous work on black-box intermediate activation reconstruction attacks [17] can be adopted.

Other Side-Channel Attack Vectors. Controlled channels are not the only means through which neuron activation patterns may be leaked. Hardware-based off-chip memory access side channels [12], [93] (or even physical bus interposition [13]), as demonstrated by Hua et al. in model stealing attacks [56], can also expose neuron activation patterns. CPU cache side channels [94], [95] may also reveal neuron access patterns, e.g., by flushing page table entries from CPU caches [9]. Recently, researchers also found that Intel TDX gives adversaries a new means to invalidate a cache line [53], which could enhance flush-based attacks [94], [95]. Concurrent work by Hornetz et al. combines controlled channels with CPU cache side channels to recover prompts by monitoring memory accesses during tokenization of prompts [31]. Spielman et al. [96], on the other hand, combine controlled channels with the single-stepping primitive against enclaves to recover model parameters.

Ciphertext side channels and floating-point timing side channels could be another potential vector for leaking activation patterns. Prior work used the former to leak DNN intelligence [97], and the latter to reverse-engineer DNNs [98]. Yan et al. combined ciphertext side channels with the attacker’s ability to relocate pages to infer partial information about a victim’s prompt [99]. Unlike our approach, their attack exploits leakage from sparse activations, rather than selective neuron accesses during sparse down projection. Further investigation of these and other side channel attack vectors remains an open direction for future research.

Other Risky Inference-Time Optimizations. Our work reveals a fundamental tension between inference-time optimizations increasingly used in LLM serving systems and their implications on input or model privacy. While we focused on sparse matrix-vector computation, any optimization that introduces input- or model-dependent behavior can potentially be exploited to leak information about user prompts or models.

Caching mechanisms such as KV cache, for example, introduce input-dependent behavior. A line of work showed that such behavior are exposed via timing side channels, which can in turn be exploited to recover input prompts [46], [54], [74], [100]. Mixture-of-Experts (MoE) models (including MoE-fied ones [101], [102]) also exhibit input-dependent behavior during expert selection. Prior work has shown that per-batch routing in MoE models can be abused to drop [103] and even steal [104], [105] tokens from other co-tenants scheduled in the same batch as the attacker. By abusing speculative decoding [106], Wei et al. showed (i) that side

channels can be used to distinguish successful and unsuccessful speculation, and (ii) that information about prompts can be leaked [107]. Hua et al. showed that, by abusing the zero-pruning optimization implemented in accelerators, the ratio between the weight and bias parameters can be revealed through side channels [56]. These works, including ours, highlight the need to reevaluate the security of LLM serving systems with modern optimizations.

X. RELATED WORK

A. Prompt Extraction Attacks

Our work relates to a broader line of work on prompt extraction attacks against LLMs, among which we highlight black-box, API-only attacks not covered in §IV. These attacks target system prompts, i.e., prompts that are prepended to user inputs by the LLM serving system, which are often considered proprietary and secret. Researchers proposed to extract system prompts via adversarial or jailbreak-style queries [108], [109], by observing the model’s output probabilities [18] or output tokens [73]. Unlike these attacks, ours requires no access to the LLM service interface, and can recover both system and user prompts, as well as the model’s response, by monitoring the victim’s memory accesses throughout prefill and decoding.

B. Confidential Machine Learning

Our work relates to a rich body of work that explores deploying DNN serving systems inside TEEs to protect models and user inputs [110]. A line of work focuses on on-device DNN inference for edge or mobile platforms [36]–[38], [40], [111]–[113]. Another line of work targets cloud scenarios, using Intel SGX [114] and, more recently, GPU TEEs [39] and NPU TEEs [115], for protecting models or user data during DNN computation. Notably, NVIDIA has introduced confidential computing capabilities in their H100 GPUs [116]. Tan et al. identified inefficiencies in encrypted memory transfers between CPU TEEs and H100 GPU TEEs, and proposed pipelining techniques to accelerate LLM inference on CPU-GPU TEEs [39]. The industry has been actively exploring deploying LLM services within TEEs as well [41]–[43], with Anthropic recently elaborating on the design principles of confidential inference systems [117]. While our work targets LLM serving within virtualization-based TEEs on the CPU side, it raises a broader question of whether these TEE-protected DNN serving systems are secure against side channel attacks, a direction we leave for future work.

XI. CONCLUSION

LLMs have traditionally been considered resilient to side-channel attacks due to their largely input-independent execution. Recent optimizations for LLM serving systems, however, challenge this assumption. Sparsity-aware optimizations in particular, such as sparse matrix-vector multiplications, introduce input-dependent neuron weight accesses. Our work shows that such dependencies leak neuron activation patterns over memory access side channels, and that these patterns can be inverted to reconstruct prompt and response tokens

with high accuracy, thereby enabling an end-to-end token extraction attack. Our attack succeeds even when the optimized LLM serving system is deployed within a TEE (e.g., an Intel TDX CVM), underscoring the need to carefully assess such optimizations in privacy-sensitive settings.

ACKNOWLEDGMENT

This material is based upon work supported by Samsung Electronics under grant nr. IO240514-09973-01, National Research Foundation (NRF) of Korea under grant nr. RS-2024-00334395, and by Institute of Information & Communications Technology Planning & Evaluation (IITP) of the Korea government's Ministry of Science and ICT (MSIT) under grant nr. RS-2024-00439762.

REFERENCES

- [1] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, R. Grosse, S. McCandlish, J. Kaplan, D. Amodei, M. Wattenberg, and C. Olah, "Toy models of superposition," *arXiv preprint arXiv:2209.10652*, 2022.
- [2] R. Bayat, A. Rahimi-Kalahroudi, M. Pezeshki, S. Chandar, and P. Vincent, "Steering large language model activations in sparse spaces," *arXiv preprint arXiv:2503.00177*, 2025.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re *et al.*, "Deja Vu: Contextual sparsity for efficient LLMs at inference time," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [5] N. Zheng, H. Jiang, Q. Zhang, Z. Han, L. Ma, Y. Yang, F. Yang, C. Zhang, L. Qiu, M. Yang *et al.*, "PIT: Optimization of dynamic sparse deep learning models via permutation invariant transformation," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [6] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a flash: Efficient large language model inference with limited memory," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [7] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2024.
- [8] Z. Xue, Y. Song, Z. Mi, L. Chen, Y. Xia, and H. Chen, "PowerInfer-2: Fast large language model inference on a smartphone," *arXiv preprint arXiv:2406.06282*, 2024.
- [9] J. Van Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, "Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution," in *Proceedings of the USENIX Security Symposium (Security)*, 2017.
- [10] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2015.
- [11] X. Zhuang, T. Zhang, and S. Pande, "HIDE: An infrastructure for efficiently protecting information leakage on the address bus," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2004.
- [12] D. Lee, D. Jung, I. T. Fang, C.-C. Tsai, and R. A. Popa, "An off-chip attack on hardware enclaves via the memory bus," in *Proceedings of the USENIX Security Symposium (Security)*, 2020.
- [13] J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, and D. Genkin, "Transparent domain extensions: Breaking Intel TEE implementations via DDR5 memory bus interposition," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2026, to appear.
- [14] W. Seo and J. Lee, "Unsupervised feature selection towards pattern discrimination power," in *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2024.
- [15] A. Mahendran and A. Vedaldi, "Understanding deep image representations by inverting them," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [16] W. Qu, Y. Zhou, Y. Wu, T. Xiao, B. Yuan, Y. Li, and J. Zhang, "Prompt inversion attack against collaborative inference of large language models," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2025.
- [17] T. Dong, Y. Meng, S. Li, G. Chen, Z. Liu, and H. Zhu, "Depth gives a false sense of privacy: LLM internal states inversion," in *Proceedings of the USENIX Security Symposium (Security)*, 2025.
- [18] J. X. Morris, W. Zhao, J. T. Chiu, V. Shmatikov, and A. M. Rush, "Language model inversion," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [19] Open Virtualization Alliance, "Linux kernel virtual machine." [Online]. Available: <https://www.linux-kvm.org>
- [20] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 2005.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.
- [22] Z. Li, C. You, S. Bhojanapalli, D. Li, A. S. Rawat, S. J. Reddi, K. Ye, F. Chern, F. Yu, R. Guo, and S. Kumar, "The lazy neuron phenomenon: On emergence of activation sparsity in Transformers," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [23] Z. Zhang, Y. Song, G. Yu, X. Han, Y. Lin, C. Xiao, C. Song, Z. Liu, Z. Mi, and M. Sun, "ReLU² wins: Discovering efficient activation functions for sparse LLMs," *arXiv preprint arXiv:2402.03804*, 2024.
- [24] S. I. Mirzadeh, K. Alizadeh-Vahid, S. Mehta, C. C. del Mundo, O. Tuzel, G. Samei, M. Rastegari, and M. Farajtabar, "ReLU strikes back: Exploiting activation sparsity in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [25] M. Federici, D. Belli, M. V. Baalen, A. Jalalirad, A. Skliar, B. Major, M. Nagel, and P. Whatmough, "Efficient LLM inference using dynamic input pruning and cache-aware masking," in *Proceedings of the Conference on Machine Learning and Systems (MLSys)*, 2025.
- [26] J. Liu, P. Ponnusamy, T. Cai, H. Guo, Y. Kim, and B. Athiwaratkun, "Training-free activation sparsity in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [27] Intel, "Intel® Software Guard Extensions programming reference," 2014. [Online]. Available: <https://www.intel.com/content/dam/develop/external/us/en/documents/329298-002-629101.pdf>
- [28] L. Wilke, J. Wichelmann, M. Morbitzer, and T. Eisenbarth, "SEVurity: No security without integrity: Breaking integrity-free memory encryption with minimal assumptions," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2020.
- [29] M. Li, Y. Zhang, Z. Lin, and Y. Solihin, "Exploiting unprotected I/O operations in AMD's secure encrypted virtualization," in *Proceedings of the USENIX Security Symposium (Security)*, 2019.
- [30] L. Wilke, F. Sieck, and T. Eisenbarth, "TDXdown: Single-stepping and instruction counting attacks against Intel TDX," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [31] T. Hornetz, H. Yavarzadeh, A. Cheu, A. Gascon, L. Gerlach, D. Moghimi, P. Schoppmann, M. Schwarz, and R. Zhang, "TDXRay: Microarchitectural side-channel analysis of Intel TDX for real-world workloads," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2026, to appear.
- [32] AMD, "AMD SEV-SNP," 2020. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>
- [33] Intel, "Intel® Trust Domain Extensions," 2022. [Online]. Available: <https://cdrdv2-public.intel.com/690419/TDX-Whitepaper-February2022.pdf>
- [34] —, "Intel® Trust Domain Extensions (Intel® TDX) module base architecture specification," 2025. [Online]. Available: <https://cdrdv2-public.intel.com/853286/intel-tdx-module-base-spec-348549006.pdf>
- [35] "QEMU system emulation user's guide." [Online]. Available: <https://www.qemu.org/docs/master/system/index.html>
- [36] F. Mo, A. S. Shamsabadi, K. Katevas, S. Demetriou, I. Leontiadis, A. Cavallaro, and H. Haddadi, "DarkneTZ: Towards model privacy at the edge using trusted execution environments," in *Proceedings of the Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2020.

- [37] S. Siby, S. Abdollahi, M. Maheri, M. Kogias, and H. Haddadi, "Guarantee: Towards attestable and private ML with CCA," in *Proceedings of the Workshop on Machine Learning and Systems (EuroMLSys)*, 2024.
- [38] M. Moon, M. Kim, J. Jung, and D. Song, "ASGARD: Protecting on-device deep neural networks with virtualization-based trusted execution environments," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.
- [39] Y. Tan, C. Tan, Z. Mi, and H. Chen, "PipeLLM: Fast and confidential large language model services with speculative pipelined encryption," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [40] X. Wang, J. Shi, Z. Zhao, Y. Yu, Z. Hua, and J. Gu, "TZ-LLM: Protecting on-device large language models with Arm TrustZone," in *Proceedings of the ACM European Conference on Computer Systems (EuroSys)*, 2026.
- [41] Google, "Enabling more private generative AI," 2024. [Online]. Available: <https://developers.googleblog.com/en/enabling-more-private-gen-ai>
- [42] Anthropic, "Confidential inference via trusted virtual machines," 2025. [Online]. Available: <https://www.anthropic.com/research/confidential-inference-trusted-vms>
- [43] Microsoft, "Confidential AI," 2023. [Online]. Available: <https://learn.microsoft.com/en-us/azure/confidential-computing/confidential-ai>
- [44] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, "Confidential LLM inference: Performance and cost across CPU and GPU TEEs," in *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2025.
- [45] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [46] Y. Z. Guanlong Wu, Zheng Zhang, "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.
- [47] Z. Gao, J. Hu, F. Guo, Y. Zhang, Y. Han, S. Liu, H. Li, and Z. Lv, "I know what you said: Unveiling hardware cache side-channels in local large language model inference," in *Proceedings of the USENIX Security Symposium (Security)*, 2025.
- [48] W.-C. Kang, D. Z. Cheng, T. Yao, X. Yi, T. Chen, L. Hong, and E. H. Chi, "Learning to embed categorical features without embedding tables for recommendation," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021.
- [49] M. Umar, A. P. Marathe, M. D. Gupta, S. J. Ghosh, G. E. Suh, and W. Xiong, "Efficient memory side-channel protection for embedding generation in machine learning," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025.
- [50] R. Russell, "virtio: Towards a de-facto standard for virtual I/O devices," *SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 95–103, Jul. 2008.
- [51] "Virtual I/O device (VIRTIO) version 1.2," 2022. [Online]. Available: <https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.pdf>
- [52] Google, "Implementing dm-verity," 2024. [Online]. Available: <https://source.android.com/docs/security/features/verifiedboot/dm-verity>
- [53] E. Aktas, C. Cohen, J. Eads, J. Forshaw, and F. Wilhelm, "Intel trust domain extensions (TDX) security review," 2023. [Online]. Available: https://services.google.com/fh/files/misc/intel_tdx_-_full_report_041423.pdf
- [54] Z. Luo, S. Shao, S. Zhang, L. Zhou, Y. Hu, C. Zhao, Z. Liu, and Z. Qin, "Shadow in the cache: Unveiling and mitigating privacy risks of KV-cache in LLM inference," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.
- [55] D. Pasquini, E. M. Kornaropoulos, and G. Ateniese, "LLMmap: Fingerprinting for large language models," in *Proceedings of the USENIX Security Symposium (Security)*, 2025.
- [56] W. Hua, Z. Zhang, and G. E. Suh, "Reverse engineering convolutional neural networks through side-channel information leaks," in *Proceedings of the Annual Design Automation Conference (DAC)*, 2018.
- [57] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn DNN architectures," in *Proceedings of the USENIX Security Symposium (Security)*, 2020.
- [58] Y. Gao, H. Qiu, Z. Zhang, B. Wang, H. Ma, A. Abudabbba, M. Xue, A. Fu, and S. Nepal, "DeepTheft: Stealing DNN model architectures through power side channel," in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [59] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [60] F. Rauscher, L. Wilke, H. Weissteiner, T. Eisenbarth, and D. Gruss, "TDXploit: Novel techniques for single-stepping and cache attacks on Intel TDX," in *Proceedings of the USENIX Security Symposium (Security)*, 2025.
- [61] A. Downey, *The little book of semaphores*. Green Tea Press, 2008, vol. 2, no. 2.
- [62] H. He and W. J. Su, "A law of next-token prediction in large language models," *Phys. Rev. E*, 2025.
- [63] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open pre-trained Transformer language models," 2022.
- [64] SpaseLLM Team, "Sparse large language models with ReLU activation," 2023. [Online]. Available: <https://huggingface.co/SpaseLLM/ReluLLaMA-7B>
- [65] NVIDIA, "Nemotron-3-8B-Base-4k," 2023. [Online]. Available: <https://huggingface.co/nvidia/nemotron-3-8b-base-4k>
- [66] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [67] Gemma Team, "Gemma: Open models based on Gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.
- [68] E. Danisman, "Skytrax airline reviews," 2019. [Online]. Available: <https://www.kaggle.com/datasets/efehandanisman/skytrax-airline-reviews>
- [69] T. Han, L. C. Adams, J.-M. Papaioannou, P. Grundmann, T. Oberhauser, A. Löser, D. Truhn, and K. K. Bressem, "MedAlpaca—an open-source collection of medical conversational AI models and training data," *arXiv preprint arXiv:2304.08247*, 2023.
- [70] I. Chalkidis, I. Androutsopoulos, and N. Aletras, "Neural legal judgment prediction in English," 2019. [Online]. Available: <https://arxiv.org/abs/1906.02059>
- [71] J. Morris, "Private prompts," 2023. [Online]. Available: https://huggingface.co/datasets/jxm/private_prompts
- [72] G. Chua, "System prompt leakage," 2024. [Online]. Available: <https://huggingface.co/datasets/gabrielchua/system-prompt-leakage>
- [73] C. Zhang, J. X. Morris, and V. Shmatikov, "Extracting prompts by inverting LLM outputs," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [74] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou, "The early bird catches the leak: Unveiling timing side channels in LLM serving systems," *arXiv preprint arXiv:2409.20002*, 2025.
- [75] B. C. Das, M. H. Amini, and Y. Wu, "System prompt extraction attacks and defenses in large language models," *arXiv preprint arXiv:2505.23817*, 2025.
- [76] A. Salemi, S. Mysore, M. Bendersky, and H. Zamani, "LaMP: When large language models meet personalization," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [77] Z. Tan, Q. Zeng, Y. Tian, Z. Liu, B. Yin, and M. Jiang, "Democratizing large language models via personalized parameter-efficient fine-tuning," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [78] J. Chen, X. Wang, R. Xu, S. Yuan, Y. Zhang, W. Shi, J. Xie, S. Li, R. Yang, T. Zhu *et al.*, "From persona to personalization: A survey on role-playing language agents," *arXiv preprint arXiv:2404.18231*, 2024.
- [79] Y. Xu, J. Zhang, A. Salemi, X. Hu, W. Wang, F. Feng, H. Zamani, X. He, and T.-S. Chua, "Personalized generation in large model era: A

- survey,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [80] G. Gerganov, “llama.cpp: LLM inference in C/C++,” 2025. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [81] “Using the Linux kernel Tracepoints.” [Online]. Available: <https://www.kernel.org/doc/Documentation/trace/tracepoints.txt>
- [82] B. Gregg, “Linux extended BPF (eBPF) tracing tools,” 2018. [Online]. Available: <http://www.brendangregg.com/ebpf.html>
- [83] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a method for automatic evaluation of machine translation,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002.
- [84] Wikimedia Foundation. Wikimedia downloads. [Online]. Available: <https://dumps.wikimedia.org>
- [85] The kernel development community, “dm-crypt,” 2024. [Online]. Available: <https://www.kernel.org/doc/html/v6.7/admin-guide/device-mapper/dm-crypt.html>
- [86] O. Goldreich and R. Ostrovsky, “Software protection and simulation on oblivious RAMs,” *Journal of the ACM (JACM)*, vol. 43, no. 3, pp. 431–473, 1996.
- [87] A. Rane, C. Lin, and M. Tiwari, “Raccoon: Closing digital side-channels through obfuscated execution,” in *Proceedings of the USENIX Security Symposium (Security)*, 2015.
- [88] K. Nayak, C. W. Fletcher, L. Ren, N. Chandran, S. V. Lokam, E. Shi, and V. Goyal, “HOP: Hardware makes obfuscation practical,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.
- [89] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: high-throughput generative inference of large language models with a single GPU,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [90] W. Lee, J. Lee, J. Seo, and J. Sim, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [91] D. Park and B. Egger, “Improving throughput-oriented LLM inference with CPU computations,” in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2024.
- [92] Z. Liu, Y. Yuan, Y. Chen, S. Hu, T. Li, and S. Wang, “DeepCache: Revisiting cache side-channel attacks in deep neural networks executables,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [93] M. Maas, E. Love, E. Stefanov, M. Tiwari, E. Shi, K. Asanovic, J. Kubiatowicz, and D. Song, “Phantom: Practical oblivious computation in a secure processor,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2013.
- [94] Y. Yarom and K. Falkner, “FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack,” in *Proceedings of the USENIX Security Symposium (Security)*, 2014.
- [95] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, “Flush+Flush: a fast and stealthy cache attack,” in *Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2016.
- [96] J. Spielman, D. Oswald, M. Ryan, and J. Van Bulck, “Activation functions considered harmful: Recovering neural network weights through controlled channels,” in *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2025.
- [97] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, “HyperTheft: Thieving model weights from TEE-shielded neural networks via ciphertext side channels,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [98] C. Gongye, Y. Fei, and T. Wahl, “Reverse-engineering deep neural networks using floating-point timing side-channels,” in *Proceedings of the ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2020.
- [99] Y. Yan, W. Huang, I. Grishchenko, G. Saileshwar, A. Mehta, and D. Lie, “Relocate-Vote: Using sparsity information to exploit ciphertext side-channels,” in *Proceedings of the USENIX Security Symposium (Security)*, 2025.
- [100] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, Q. Guo, and X. Hu, “InputSnatch: Stealing input in LLM services via timing side-channel attacks,” *arXiv preprint arXiv:2411.18191*, 2024.
- [101] Z. Zhang, Y. Lin, Z. Liu, P. Li, M. Sun, and J. Zhou, “MoEfication: Transformer feed-forward layers are mixtures of experts,” in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.
- [102] F. Szatkowski, B. Wójcik, M. Piórczyński, and S. Scardapane, “Exploiting activation sparsity with dense to dynamic-k mixture-of-experts conversion,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [103] J. Hayes, I. Shumailov, and I. Yona, “Buffer overflow in mixture of experts,” *arXiv preprint arXiv:2402.05526*, 2024.
- [104] I. Yona, I. Shumailov, J. Hayes, and N. Carlini, “Stealing user prompts from mixture of experts,” *arXiv preprint arXiv:2410.22884*, 2024.
- [105] R. Ding, T. Xu, X. Shen, A. A. Ding, and Y. Fei, “MoEcho: Exploiting side-channel attacks to compromise user privacy in mixture-of-experts LLMs,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025.
- [106] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from Transformers via speculative decoding,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [107] J. Wei, A. Abdurrazzag, T. Zhang, A. Muursepp, and G. Saileshwar, “Privacy risks of speculative decoding in large language models,” *arXiv preprint arXiv:2411.01076*, 2024.
- [108] Y. Zhang, N. Carlini, and D. Ippolito, “Effective prompt extraction from language models,” in *Proceedings of the Conference on Language Modeling (COLM)*, 2024.
- [109] B. Hui, H. Yuan, N. Gong, P. Burlina, and Y. Cao, “PLeak: Prompt leaking attacks against large language model applications,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [110] F. Mo, Z. Tarkhani, and H. Haddadi, “Machine learning with confidential computing: A systematization of knowledge,” *ACM Computing Survey (CSUR)*, vol. 56, no. 11, Jun. 2024.
- [111] Z. Sun, R. Sun, C. Liu, A. R. Chowdhury, L. Lu, and S. Jha, “ShadowNet: A secure and efficient on-device model inference system for convolutional neural networks,” in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2023.
- [112] Z. Zhang, C. Gong, Y. Cai, Y. Yuan, B. Liu, D. Li, Y. Guo, and X. Chen, “No privacy left outside: On the (in-)security of TEE-shielded DNN partition for on-device ML,” in *Proceedings of the IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [113] T. Shen, J. Qi, J. Jiang, X. Wang, S. Wen, X. Chen, S. Zhao, S. Wang, L. Chen, X. Luo, F. Zhang, and H. Cui, “SOTER: Guarding black-box inference for general neural networks at the edge,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2022.
- [114] T. Lee, Z. Lin, S. Pushp, C. Li, Y. Liu, Y. Lee, F. Xu, C. Xu, L. Zhang, and J. Song, “Occlumency: Privacy-preserving remote deep-learning inference using SGX,” in *Proceedings of the Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2019.
- [115] E. Feng, D. Feng, D. Du, Y. Xia, and H. Chen, “sNPU: Trusted execution environments on integrated NPUs,” in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [116] NVIDIA, “Confidential Compute on NVIDIA Hopper H100,” 2023. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf>
- [117] Pattern Labs and Anthropic, “Confidential inference systems: Design principles and security risks,” 2025. [Online]. Available: https://assets.anthropic.com/m/c52125297b85a42/original/Confidential_Inference_Paper.pdf